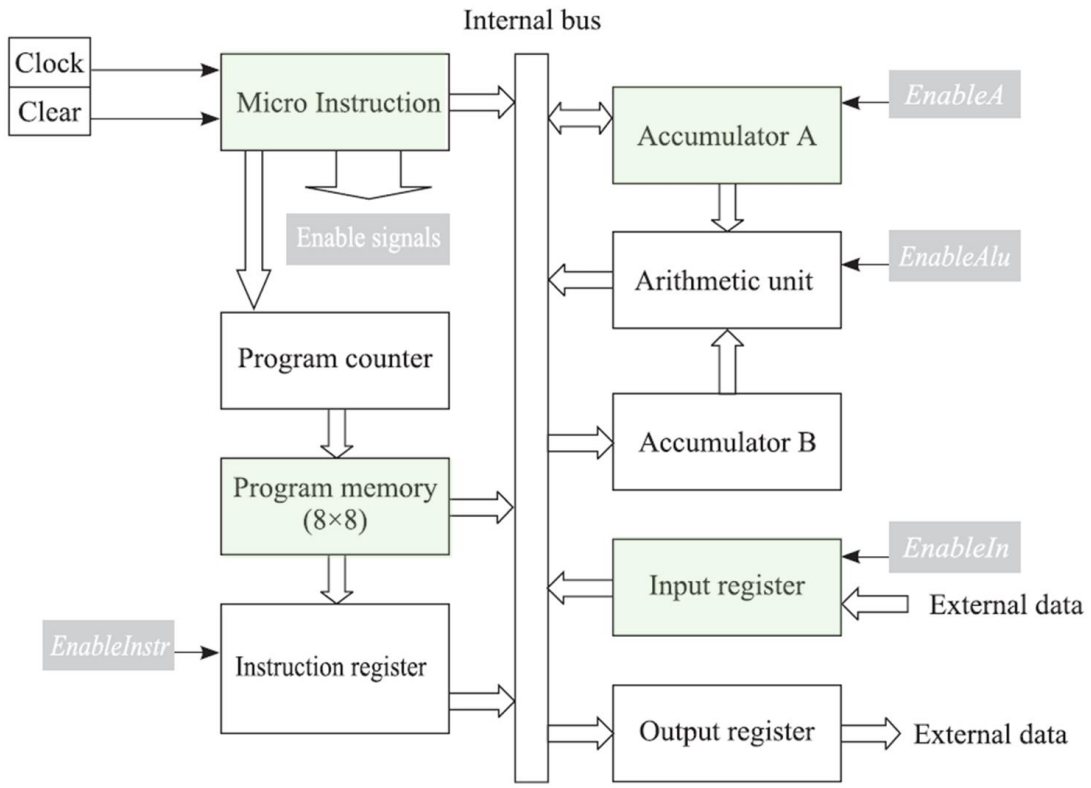
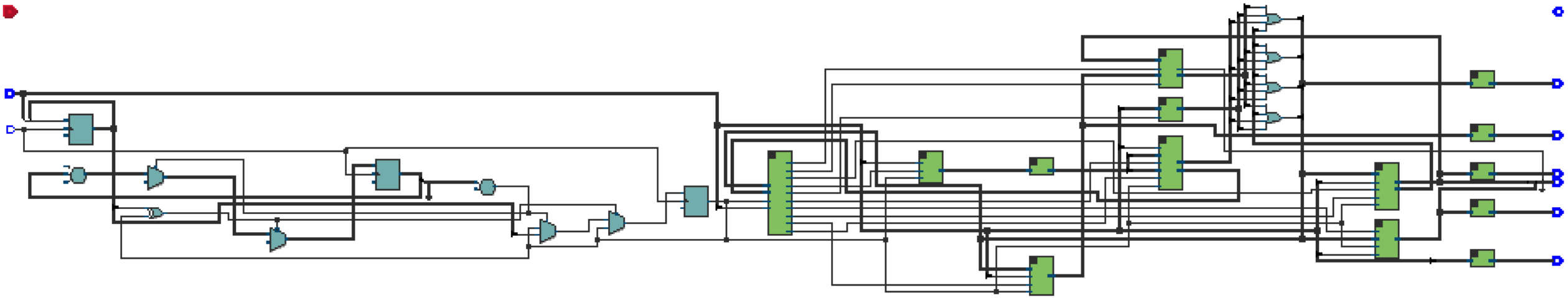
Advanced Digital Design Lab 7 Microprocessor

Diego Deoliveira

11/9/2026



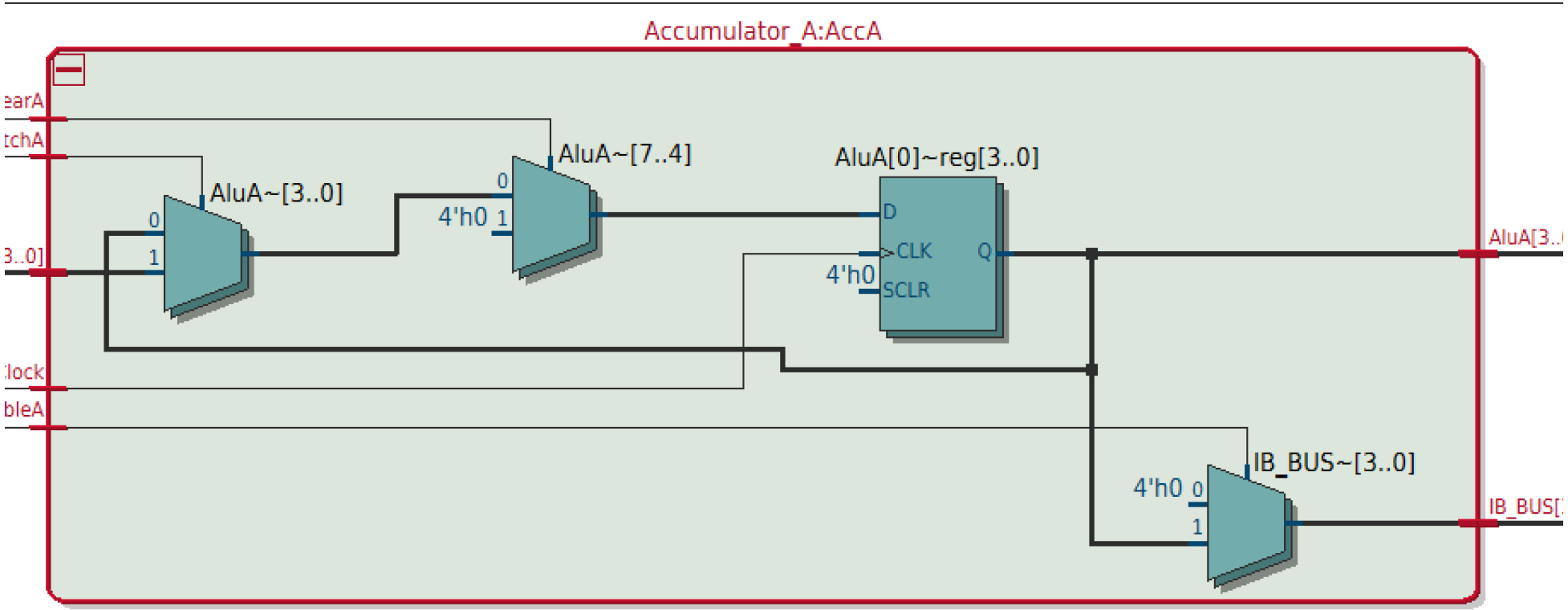
Top Level RTL Viewer



Accumulator A Code

```
1
2   `default_nettype none
3
4   module Accumulator_A #
5   (
6       parameter N = 4
7   )
8   (
9       input MainClock,
10      input ClearA,
11      input LatchA,
12      input EnableA,
13      input [N-1:0] A,
14      output [N-1:0] IB_BUS,
15      output reg [N-1:0] AluA
16  );
17
18      // Sequential Logic
19      always @(posedge MainClock) begin
20          if(ClearA)
21              AluA <= {N{1'b0}};
22          else begin
23              if(LatchA)
24                  AluA <= A;
25          end
26      end
27
28      // Combinational Logic
29      assign IB_BUS = (EnableA) ? AluA : {N{1'bz}};
30
31  endmodule
32
33  `default_nettype wire
```

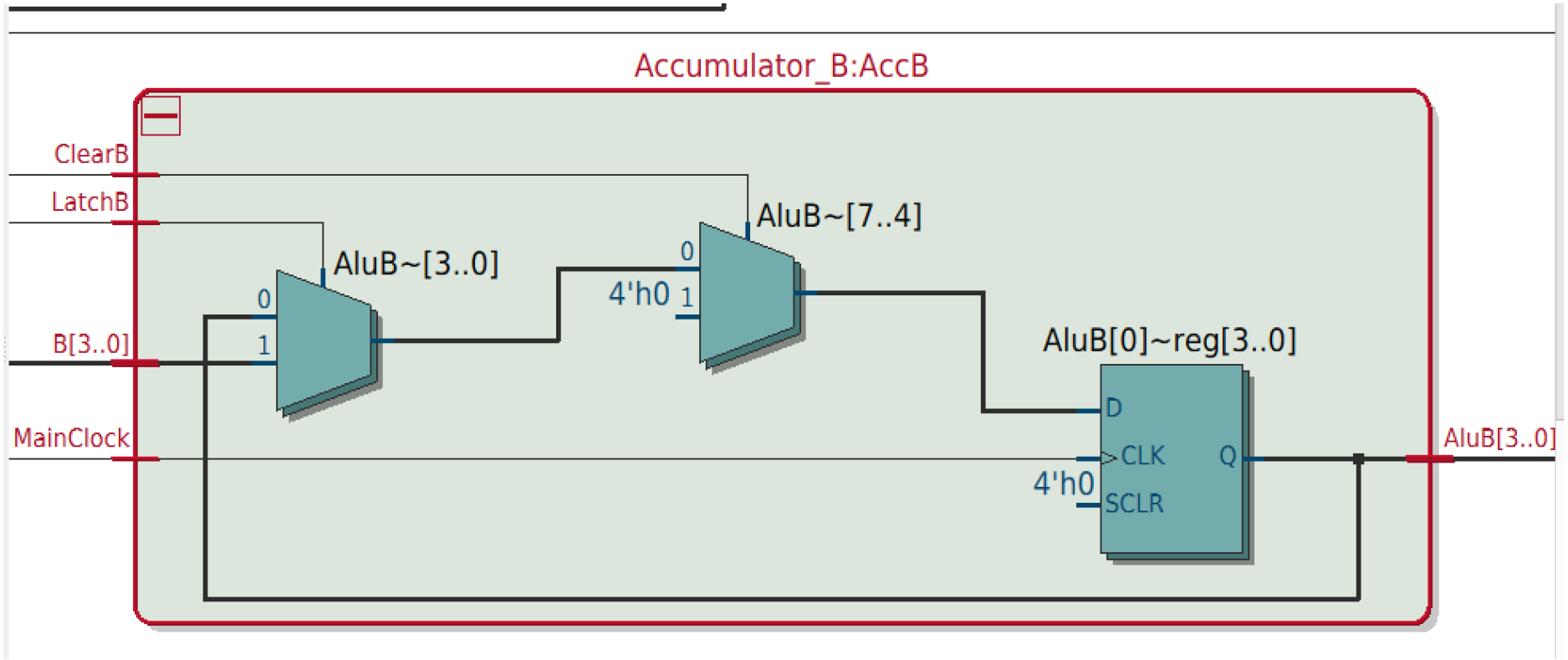
Accumulator A RTL Viewer



Accumulator B Code

```
1   `default_nettype none
2
3   module Accumulator_B #
4   (
5       parameter N = 4
6   )
7   (
8       input MainClock,
9       input ClearB,
10      input LatchB,
11      input [N-1:0] B,
12      output reg [N-1:0] AluB
13  );
14
15      // Store the second number for ADD or SUB.
16      always @(posedge MainClock) begin
17          if (ClearB)
18              AluB <= {N{1'b0}};
19          else if (LatchB)
20              AluB <= B;
21      end
22
23  endmodule
24
25  `default_nettype wire
26
```

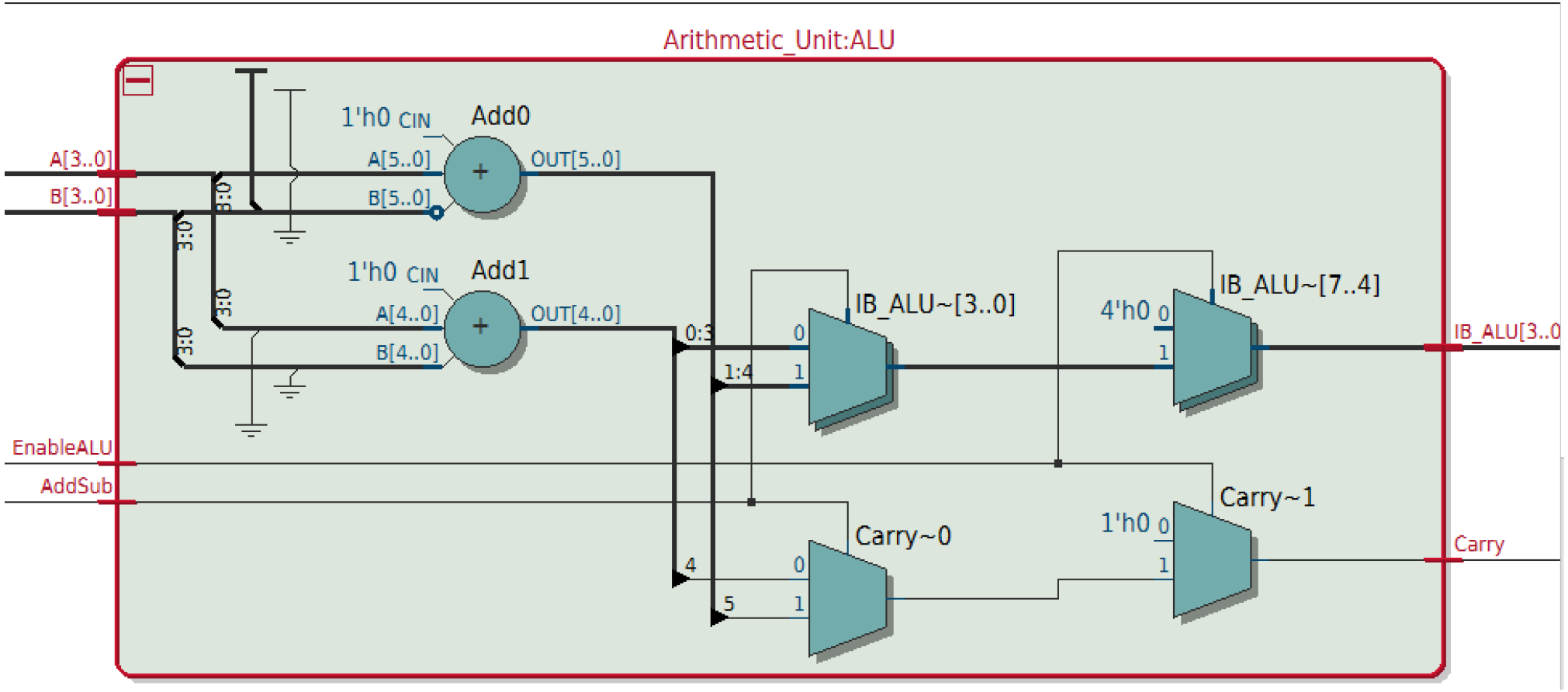
Accumulator B RTL Viewer



ALU Code

```
1   `default_nettype none
2
3   module Arithmetic_Unit #
4   (
5       parameter N = 4
6   )
7   (
8       input EnableALU,
9       input AddSub,
10      input [N-1:0] A,
11      input [N-1:0] B,
12      output reg Carry,
13      output reg [N-1:0] IB_ALU
14  );
15
16      // AddSub = 0: A + B
17      // AddSub = 1: A - B
18      // The ALU output is zero when the ALU is not enabled.
19      always @(*) begin
20          Carry = 1'b0;
21          IB_ALU = {N{1'b0}};
22
23          if (EnableALU) begin
24              if (AddSub)
25                  {Carry, IB_ALU} = {1'b0, A} - {1'b0, B};
26              else
27                  {Carry, IB_ALU} = {1'b0, A} + {1'b0, B};
28          end
29      end
30
31  endmodule
32
33  `default_nettype wire
34
```

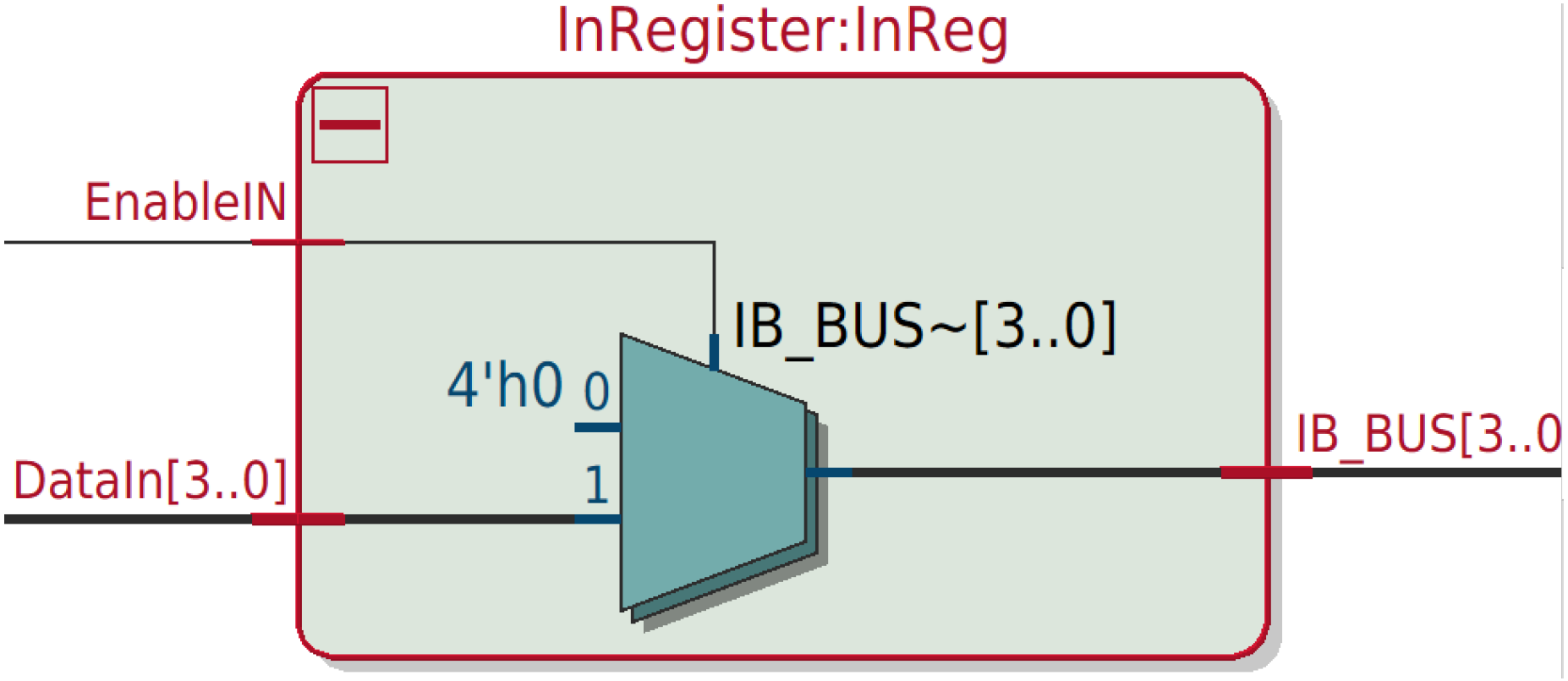
ALU RTL Viewer



Input Register Code

```
1   `default_nettype none
2
3   module InRegister #
4   (
5       parameter N = 4
6   )
7   (
8       input EnableIN,
9       input [N-1:0] DataIn,
10      output [N-1:0] IB_BUS
11  );
12
13      // Make the switches available only when EnableIN is on.
14      assign IB_BUS = EnableIN ? DataIn : {N{1'b0}};
15
16  endmodule
17
18  `default_nettype wire
19
```

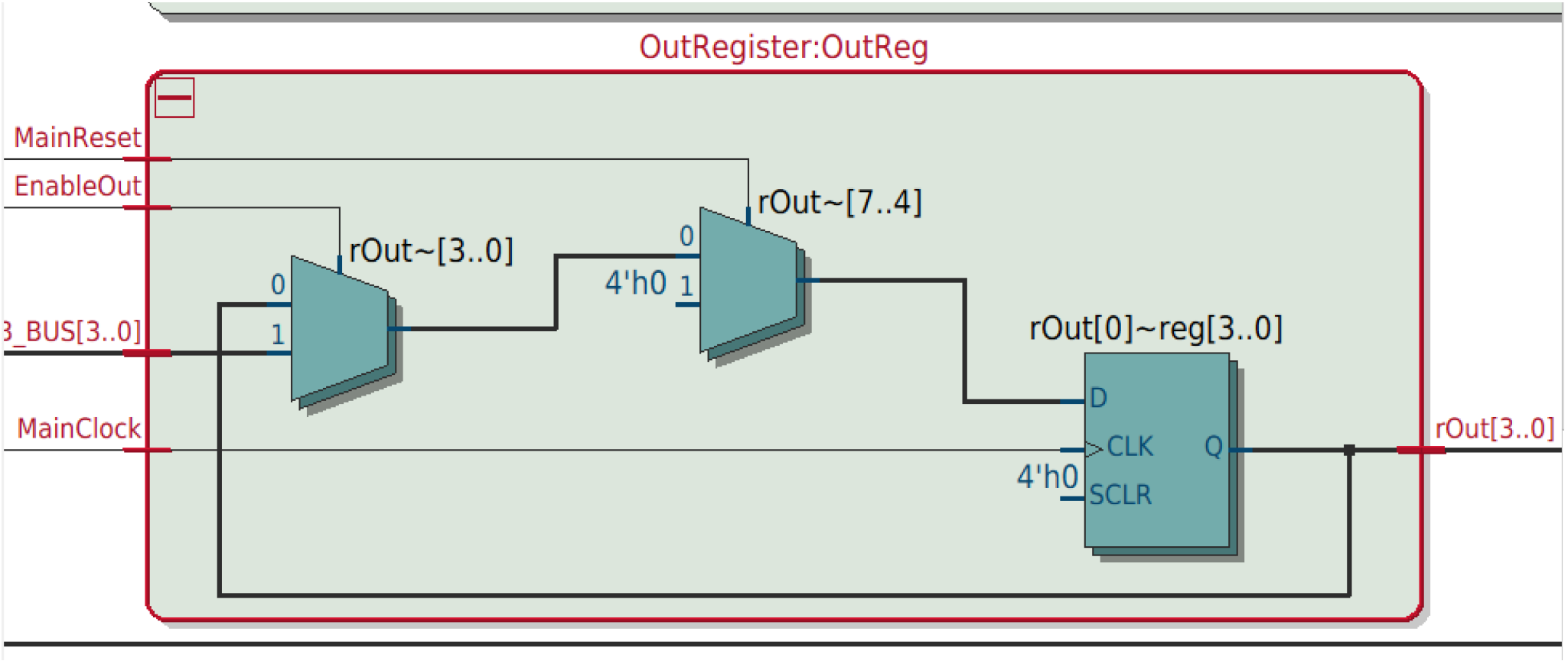
Input Register RTL Viewer



Output Register Code

```
1   `default_nettype none
2
3   module OutRegister #
4   (
5       parameter N = 4
6   )
7   (
8       input MainClock,
9       input MainReset,
10      input EnableOut,
11      input [N-1:0] IB_BUS,
12      output reg [N-1:0] rOut
13  );
14
15      // Save the value that should be displayed on the LEDs/HEX5.
16      always @(posedge MainClock) begin
17          if (MainReset)
18              rOut <= {N{1'b0}};
19          else if (EnableOut)
20              rOut <= IB_BUS;
21      end
22
23  endmodule
24
25  `default_nettype wire
26
```

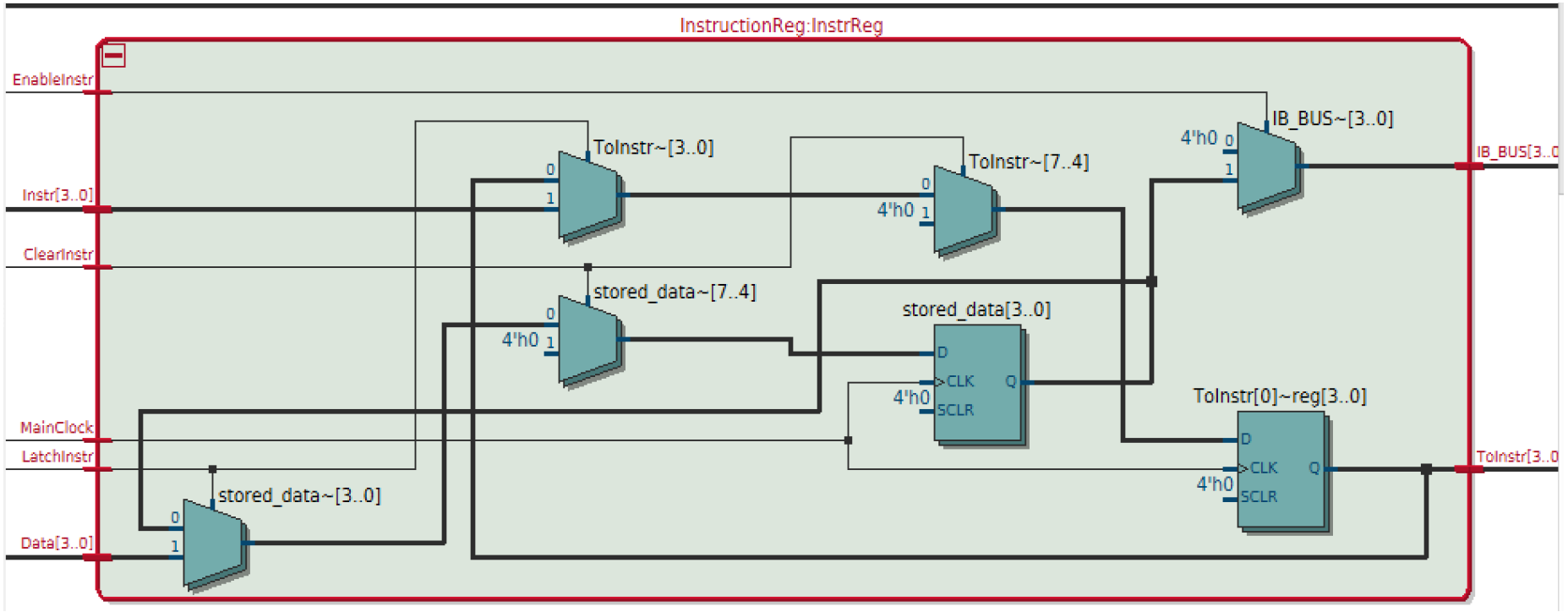
Output Register RTL Viewer



Instruction Register Code

```
1   `default_nettype none
2
3   module InstructionReg #
4   (
5       parameter N = 4
6   )
7   (
8       input MainClock,
9       input ClearInstr,
10      input LatchInstr,
11      input EnableInstr,
12      input [N-1:0] Data,
13      input [N-1:0] Instr,
14      output reg [N-1:0] ToInstr,
15      output [N-1:0] IB_BUS
16  );
17
18      reg [N-1:0] stored_data;
19
20      // The top nibble is the opcode; the bottom nibble is its value.
21      always @(posedge MainClock) begin
22          if (ClearInstr) begin
23              ToInstr <= {N{1'b0}};
24              stored_data <= {N{1'b0}};
25          end
26          else if (LatchInstr) begin
27              ToInstr <= Instr;
28              stored_data <= Data;
29          end
30      end
31
32      // Make the stored number available during execution.
33      assign IB_BUS = EnableInstr ? stored_data : {N{1'b0}};
34
35  endmodule
36
37  `default_nettype wire
38
```

Instruction Register RTL Viewer



Program Memory (ROM) Code

The ROM stores the assembly instructions that tell the microprocessor what to do. For this lab, I chose a simple program that demonstrates all five required instructions: LDA, ADD, SUB, IN, and OUT.

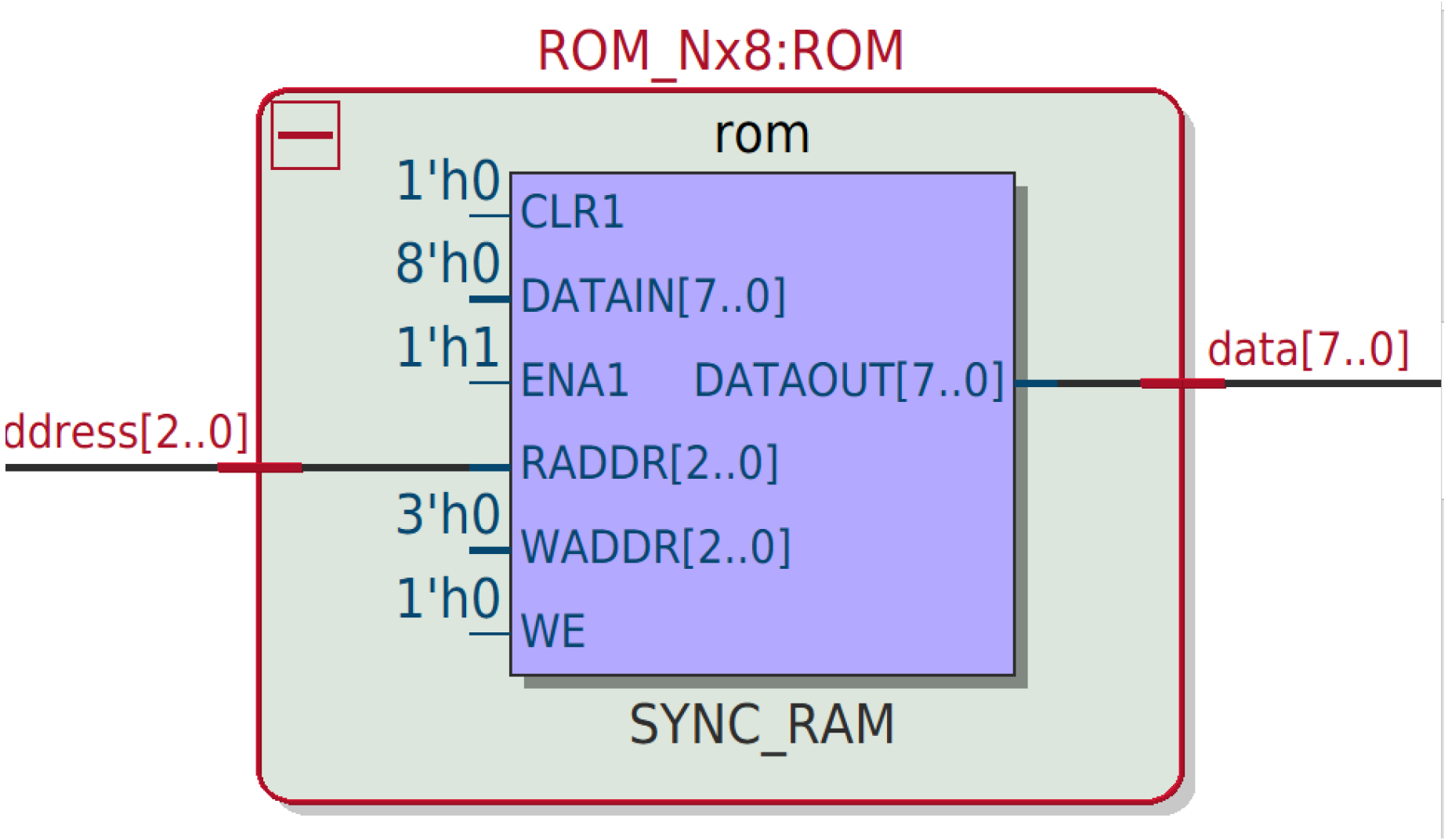
I started by loading the number 5 into Accumulator A, then adding 2 to get 7. I used the OUT instruction to display that result before subtracting 3 to get 4. I then used another OUT instruction to display 4. Finally, I included the IN instruction to read a number from the FPGA switches, followed by OUT to display that number.

I chose these instructions because they make it easy to see each operation working on the FPGA board. The numbers are small, so the results are easy to follow and verify. Using OUT multiple times also allows us to check the results at different points in the program.

The code works by storing each instruction in a different ROM address. Each eight-bit value contains a four-bit instruction code and a four-bit number. For example, `8'h55` means LDA 5, while `8'h12` means ADD 2. The Program Counter selects which ROM address to read, allowing the microprocessor to execute the instructions in order.

```
1   `default_nettype none
2
3   module ROM_Nx8 #
4   (
5       parameter N = 8
6   )
7   (
8       input [$clog2(N)-1:0] address,
9       output reg [7:0] data
10  );
11
12      // Upper 4 bits = instruction, lower 4 bits = number.
13      // LDA = 5, ADD = 1, SUB = 2, OUT = 3, IN = 4, NOP = 0.
14      (* ramstyle = "logic" *) reg [7:0] rom [0:N-1];
15
16      initial begin
17          rom[0] = 8'h55;  // LDA 5   -> A = 5
18          rom[1] = 8'h12;  // ADD 2   -> A = 7
19          rom[2] = 8'h30;  // OUT     -> show 7
20          rom[3] = 8'h23;  // SUB 3   -> A = 4
21          rom[4] = 8'h30;  // OUT     -> show 4
22          rom[5] = 8'h40;  // IN      -> A = SW[3:0]
23          rom[6] = 8'h30;  // OUT     -> show switch value
24          rom[7] = 8'h00;  // NOP     -> do nothing
25      end
26
27      // Read the instruction at the current address.
28      always @(*) begin
29          data = rom[address];
30      end
31
32  endmodule
33
34  `default_nettype wire
35
```

Program Memory (ROM) RTL Viewer



Program Memory (ROM) Cont.

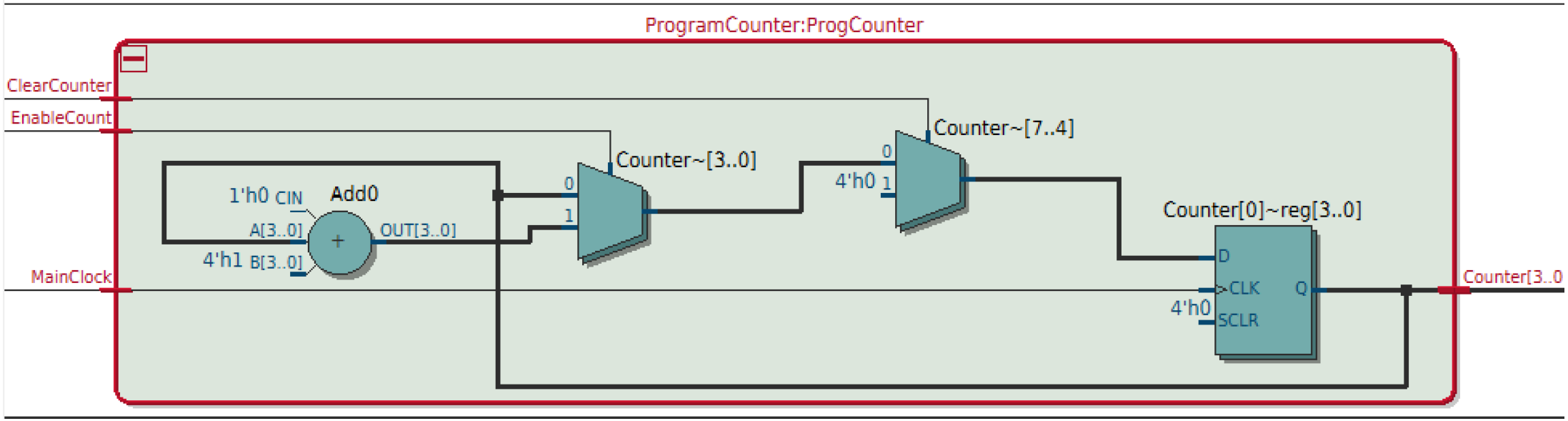
ROM Instructions Used

Address	Hex Value	Instruction	Purpose
0	`8'h55`	LDA 5	Load 5 into Accumulator A
1	`8'h12`	ADD 2	Add 2 to get 7
2	`8'h30`	OUT	Display 7
3	`8'h23`	SUB 3	Subtract 3 to get 4
4	`8'h30`	OUT	Display 4
5	`8'h40`	IN	Read the FPGA switches
6	`8'h30`	OUT	Display the switch value
7	`8'h00`	NOP	Do nothing

Program Counter Code

```
1  `default_nettype none
2
3  module ProgramCounter #
4  (
5      parameter N = 4
6  )
7  (
8      input MainClock,
9      input EnableCount,
10     input ClearCounter,
11     output reg [N-1:0] Counter
12 );
13
14     // Move to the next instruction after each fetch.
15     always @(posedge MainClock) begin
16         if (ClearCounter)
17             Counter <= {N{1'b0}};
18         else if (EnableCount)
19             Counter <= Counter + 1'b1;
20     end
21
22 endmodule
23
24 `default_nettype wire
25
```

Program Counter RTL Viewer



FSM Controller Code

```
1
2   `default_nettype none
3
4   module FSM_MicroInstr #
5   (
6      parameter N = 4
7   )
8   (
9      input clk,
10     input reset,
11     input [N-1:0] IB_BUS,
12
13     output reg LatchA,
14     output reg EnableA,
15     output reg LatchB,
16     output reg EnableALU,
17     output reg AddSub,
18     output reg EnableIN,
19     output reg EnableOut,
20     output reg LoadInstr,
21     output reg EnableInstr,
22     input [N-1:0]  ToInstr,
23     output reg EnableCount
24  );
25
26     reg [2:0] state, next_state;
27
28     // States
29     localparam [2:0] IDLE = 3'd0, PHASE_1 = 3'd1, PHASE_2 = 3'd2, PHASE_3 = 3'd3, PHASE_4 = 3'd4;
30
31     // 1. State Register (sequential)
32     always @(posedge clk) begin
33        if(reset)
34           state <= IDLE;
35        else
36           state <= next_state;
37     end
38
```

```
38
39     // 2. Next-State Logic (combinational)
40     always @(*) begin
41
42        next_state = state; // default: hold state
43
44        case(state)
45
46           IDLE:
47              begin
48                 // Move to FETCH
49                 next_state = PHASE_1;
50              end
51
52           // FETCH Instruction
53           PHASE_1:
54              begin
55                 // Move to DECODE
56                 next_state = PHASE_2;
57              end
58
59           // DECODE Instruction
60           PHASE_2:
61              begin
62                 // Move to EXECUTE A
63                 next_state = PHASE_3;
64              end
65
66           // EXECUTE A Instruction
67           PHASE_3:
68              begin
69                 // Move to EXECUTE B
70                 next_state = PHASE_4;
71              end
72
73           // EXECUTE B Instruction
74           PHASE_4:
75              begin
76                 // Move to FETCH
```

FSM Controller Code Cont.

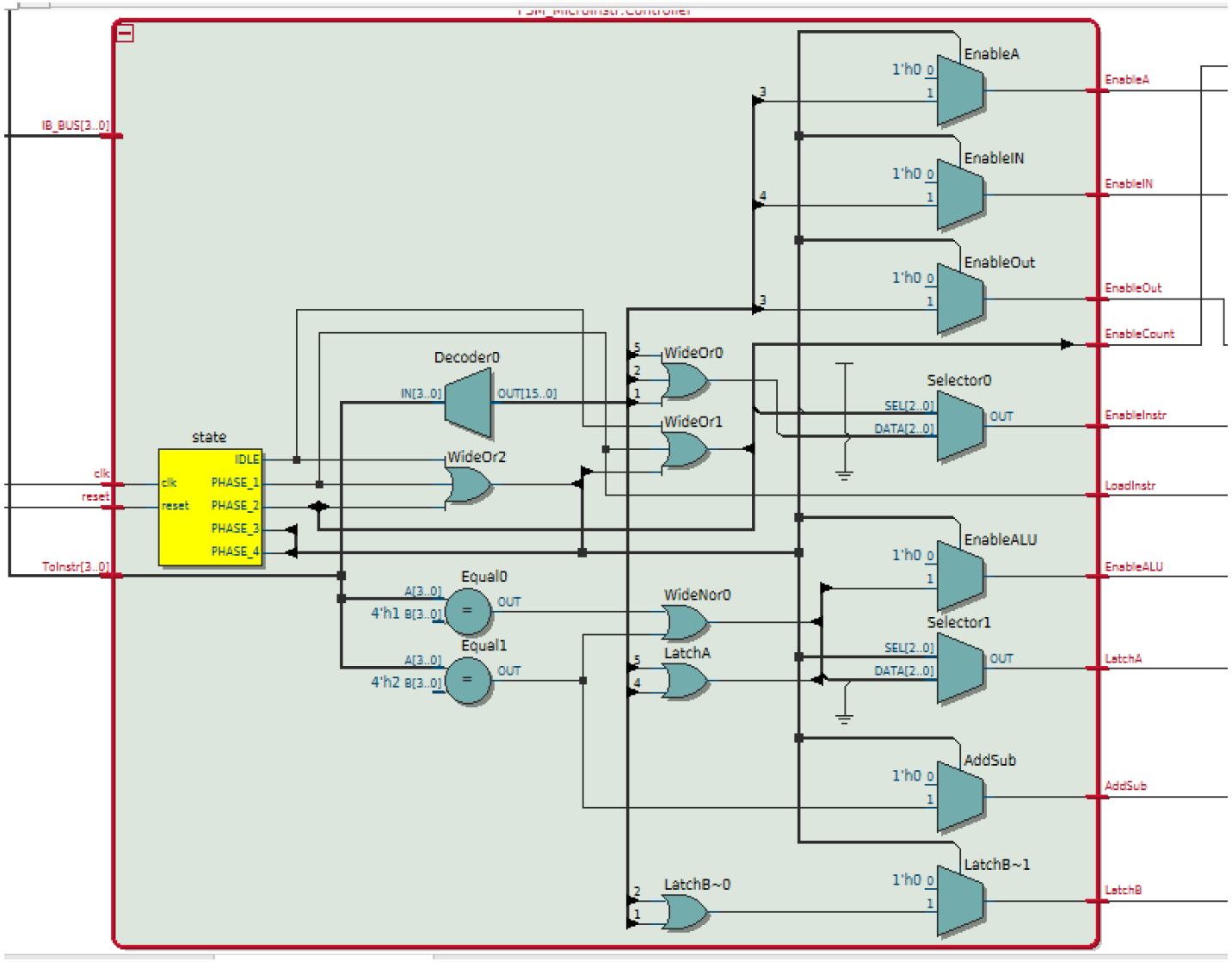
```
74          PHASE_4:
75             begin
76                // Move to FETCH
77                next_state = PHASE_1;
78             end
79
80          default: ;// None
81
82       endcase
83    end
84
85    // 3. Output Logic (combinational)
86
87    always @(*) begin
88
89       // default all signals are zero
90       LoadInstr   = 1'b0;
91       EnableInstr = 1'b0;
92       LatchB      = 1'b0;
93       LatchA      = 1'b0;
94       EnableALU   = 1'b0;
95       EnableCount = 1'b0;
96       AddSub      = 1'b0;
97       EnableIN    = 1'b0;
98       EnableA     = 1'b0;
99       EnableOut   = 1'b0;
100
101      case(state)
102
103         IDLE:
104            begin
105               // Initialize things if needed
106            end
107
108         // FETCH: load the ROM instruction into the register.
109         PHASE_1:
110            begin
111               LoadInstr = 1'b1;
```

```
111               LoadInstr = 1'b1;
112            end
113
114         // DECODE: advance to the next ROM address.
115         PHASE_2:
116            begin
117               EnableCount = 1'b1;
118               EnableInstr = 1'b1;
119            end
120
121         // EXECUTE A: get an operand or perform a simple instruction.
122         PHASE_3:
123            begin
124               case (ToInstr)
125                  4'b0001: begin // ADD: load number into B
126                     LatchB = 1'b1;
127                     EnableInstr = 1'b1;
128                  end
129                  4'b0010: begin // SUB: load number into B
130                     LatchB = 1'b1;
131                     EnableInstr = 1'b1;
132                  end
133                  4'b0011: begin // OUT: copy A to output register
134                     EnableOut = 1'b1;
135                     EnableA = 1'b1;
136                  end
137                  4'b0100: begin // IN: load switches into A
138                     LatchA = 1'b1;
139                     EnableIN = 1'b1;
140                  end
141                  4'b0101: begin // LDA: load immediate value into A
142                     LatchA = 1'b1;
143                     EnableInstr = 1'b1;
144                  end
145                  default: ; // NOP
146               endcase
147            end
148
```

FSM Controller Code Cont.

```
148
149             // EXECUTE B: finish an ADD or SUB.
150             PHASE_4:
151                 begin
152                     case (ToInstr)
153                         4'b0001: begin // ADD
154                             LatchA = 1'b1;
155                             EnableALU = 1'b1;
156                         end
157                         4'b0010: begin // SUB
158                             LatchA = 1'b1;
159                             EnableALU = 1'b1;
160                             AddSub = 1'b1;
161                         end
162                         default: ; // Other instructions finished in PHASE_3
163                     endcase
164                 end
165
166         endcase
167     end
168
169 endmodule
170
171 `default_nettype wire
172
```

FSM Controller RTL Viewer



Flow Summary

Flow Summary

<<Filter>>

Flow Status	Successful - Tue Nov 10 16:52:22 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	main
Top-level Entity Name	main
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	121 / 49,760 (< 1 %)
Total registers	49
Total pins	99 / 360 (28 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

Verification and Validation

To test the microprocessor, I programmed the DE10-Lite FPGA and manually stepped through the instructions using SW9 as the clock. I checked the values displayed on the seven-segment displays to see if they matched the expected results. The program was designed to load 5, add 2 to get 7, subtract 3 to get 4, and finally read a value from the FPGA switches. With the input switches set to binary 1001, the final output should be 9. The FPGA demonstration video shows the instruction sequence and allows the displayed results to be compared with the expected values.